

به نام خدا



برنامه سازی پیشرفته

دانشگاه شهید بهشتی - دانشکده مهندسی و علوم کامپیوتر

دکتر مجتبی وحیدی اصل

ارث بری - مقیدسازی - پلی مورفیسم

سارا شیری

فهرست مطالب

1. بازنویسی متدهای ابرکلاس در زیرکلاس
2. مقایسه سربارگذاری متدها و بازنویسی متدها
3. کلاس Object در جاوا
4. پلی مورفیسم
5. مقیدسازی
6. تبدیل اشیا رو به بالا / رو به پایین
7. عملگر instanceof

اعلان یک زیرکلاس

یک زیرکلاس فیلدهای داده‌ای و متدها را از ابرکلاس یا همان پدرش ارث‌بری می‌کند. یک فرزند علاوه بر ارث‌بری فیلدهای پدر می‌تواند فیلدهای داده‌ای جدید داشته باشد، متدهای جدید داشته باشد و متدهای ابرکلاس را بازنویسی (override) کند.

بازنویسی متدهای ابرکلاس در زیرکلاس‌ها

همان‌طور که در اسلاید قبل گفتیم، یک زیرکلاس متدهای پدرش یا همان ابرکلاس را ارث‌بری می‌کند. بعضی وقت‌ها این زیرکلاس نیاز دارد پیاده‌سازی یک متد تعریف‌شده در ابرکلاسش را تغییر دهد. به این کار به اصطلاح **بازنویسی متد** یا **method overriding** می‌گوییم.

به عبارت دیگر، هرگاه متدی در زیرکلاس بخواهد پیاده‌سازی خاص خود از متد ارث برده‌شده از ابرکلاس را داشته باشد، از بازنویسی استفاده می‌کند.

مزیت بازنویسی متد در پلی مورفیسم زمان اجرا می‌باشد.

قوانین بازنویسی متد:

متد باید هم‌نام با متد کلاس والد باشد.

متد باید پارامترهای مشابهی با کلاس والد داشته باشد.

رابطه وراثت برقرار باشد.

```
public class Circle extends GeometricObject {
    // Other methods are omitted

    /** Override the toString method defined in GeometricObject */
    public String toString() {
        return super.toString() + "\nradius is " + radius;
    }
}

// from class GeometricObject
/** Return a string representation of this object */
public String toString() {
    return "created on " + dateCreated +
        "\ncolor: " + color + " and filled: " + filled;
}
```


اگر از بازنویسی متد استفاده نکنیم

مسئله‌ای که پیش می‌آید: اینکه بخواهیم یک پیاده‌سازی ویژه از متد `run()` در زیرکلاس دوچرخه داشته باشیم، باید این متد را که از کلاس وسیله نقلیه به ارث رسیده است بازنویسی کنیم.

```
class Vehicle {  
    void run() {  
        System.out.println("Vehicle is running");  
    }  
}  
  
class Bike extends Vehicle {  
    public static void main(String args[]) {  
        Bike obj = new Bike();  
        obj.run();  
    }  
}
```

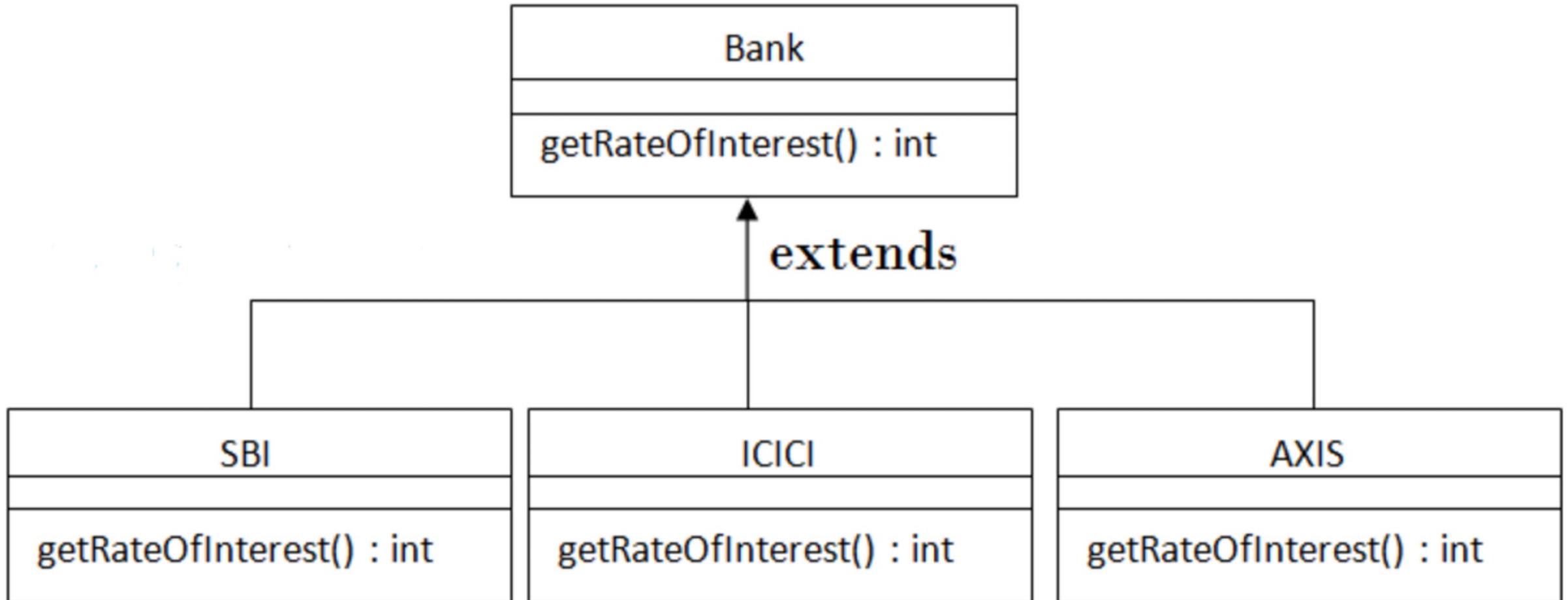
مثال استفاده از بازنویسی متد

نکته: باید حواسمان باشد که نام و نوع پارامتر متد بازنویسی شده باید با متد ابرکلاس یکی باشد.

```
class Vehicle {  
    void run() {  
        System.out.println("Vehicle is running");  
    }  
}  
  
class Bike2 extends Vehicle {  
    void run() {  
        System.out.println("Bike is running safely");  
    }  
  
    public static void main(String args[]) {  
        Bike2 obj = new Bike2();  
        obj.run();  
    }  
}
```

مثالی از بازنویسی متد - سود بانکی

شرح مسئله: کلاسی به نام Bank داریم که نرخ سود سپرده را برمی‌گرداند. اما این نرخ سود در هر بانک با بانک دیگر متفاوت است. فرض کنید سه بانک داریم به نام‌های ICIC , SBI و AXIS که به ترتیب نرخ سود 8% ، 7% و 9% برمی‌گردانند.



```
class Bank {  
    int getRateOfInterest() {  
        return 0;  
    }  
}
```

```
class SBI extends Bank {  
    int getRateOfInterest() {  
        return 8;  
    }  
}
```

```
class ICICI extends Bank {  
    int getRateOfInterest() {  
        return 7;  
    }  
}
```

```
class AXIS extends Bank {  
    int getRateOfInterest() {  
        return 9;  
    }  
}
```

```
class Test2 {  
    public static void main(String args[]) {  
        SBI s = new SBI();  
        ICICI i = new ICICI();  
        AXIS a = new AXIS();  
  
        System.out.println("SBI Rate of Interest: " + s.getRateOfInterest());  
        System.out.println("ICICI Rate of Interest: " + i.getRateOfInterest());  
        System.out.println("AXIS Rate of Interest: " + a.getRateOfInterest());  
    }  
}
```

نکات بازنویسی متد

چرا یک متد استاتیک قابل بازنویسی نمی‌باشد؟

چون متد استاتیک به یک کلاس مقید شده است نه به اشیای کلاس. اما یک متد نمونه به یک شی از کلاس مقید شده است. متدها و فیلدهای استاتیک متعلق به تعریف کلاس در حافظه ایستا هستند که در زمان اجرا قابل تغییر نمی‌باشند. اما متدهای نمونه در حافظه heap ایجاد می‌شوند و امکان بازنویسی یا تغییر آنها وجود دارد.

آیا می‌توانیم متد main در جاوا را بازنویسی کنیم؟

جواب مشخصا خیر است چون متد main به صورت استاتیک تعریف می‌شود.

نکته: یک متد نمونه تنها در صورت در دسترس بودن، قابل بازنویسی می‌باشد. بنابراین یک متد private نمی‌تواند بازنویسی شود، چون در خارج از کلاس خود قابل دسترسی نمی‌باشد. اگر متد تعریف شده در یک زیرکلاس همانام با متدی در ابرکلاسش باشد که private تعریف شده است، این دو متد کاملاً بی‌ربط با یکدیگر در نظر گرفته می‌شوند. مانند متد نمونه، یک متد استاتیک نیز می‌تواند ارث بری شود.

با این حال، یک متد استاتیک نمی‌تواند بازنویسی شود. اگر متد استاتیک تعریف شده در یک ابرکلاس در یک زیرکلاسش بازتعریف شود، متد تعریف شده در ابرکلاس پنهان (پوشیده) خواهد شد.

overloading در مقایسه با overriding

سه تفاوت اصلی میان بازنویسی و سربارگذاری متدها وجود دارد:

بازنویسی متد به برنامه‌نویس امکان می‌دهد پیاده‌سازی خاص متد ارث‌بری‌شده از کلاس والد را ایجاد کند و همواره در دو کلاسی انجام می‌شود که رابطه ارث‌بری میان آنها وجود دارد و لازم به ذکر است که پارامترها باید یکسان باشد.

ولی سربارگذاری متد به هدف افزایش خوانایی برنامه‌ها ایجاد می‌شود و درون یک کلاس انجام می‌شود و برخلاف بازنویسی، پارامترها باید متفاوت باشند.

//Overriding

```
public class Test {  
    public static void main(String[] args) {  
        Aa a = new A();  
        a.p(10);  
        a.p(10.0);  
    }  
}  
  
class B {  
    public void p(double i) {  
        System.out.println(i * 2);  
    }  
}  
  
class A extends B {  
    //This method overrides the method in B  
    public void p(double i) {  
        System.out.println(i);  
    }  
}
```

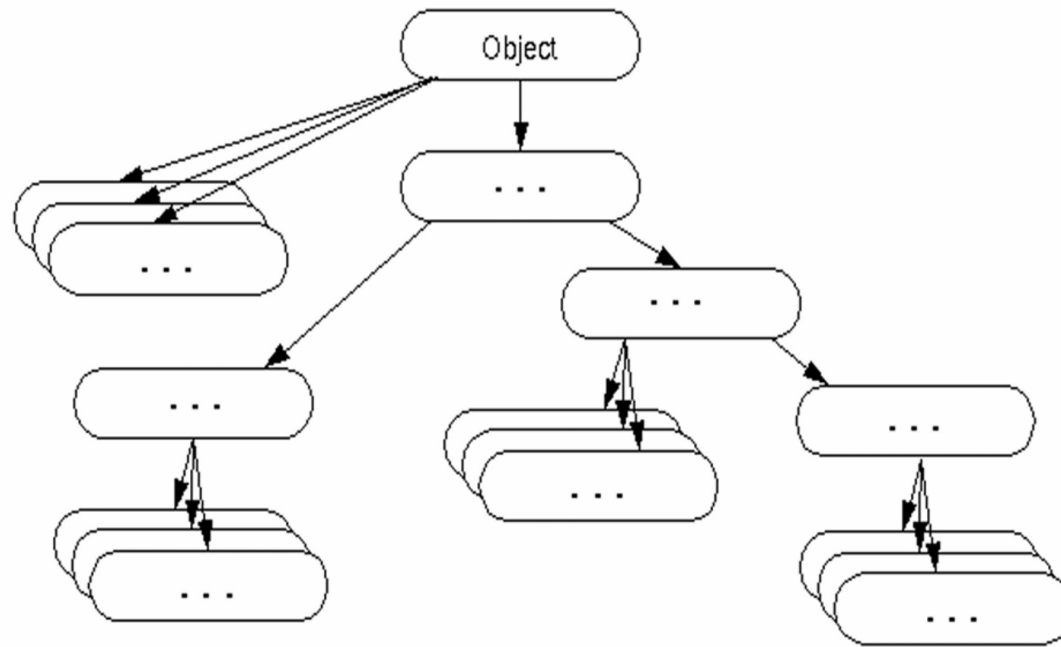
//Overloading

```
public class Test {  
    public static void main(String[] args) {  
        Aa a = new A();  
        a.p(10);  
        a.p(10.0);  
    }  
}  
  
class B {  
    public void p(double i) {  
        System.out.println(i * 2);  
    }  
}  
  
class A extends B {  
    //This method overloads the method in B  
    public void p(int i) {  
        System.out.println(i);  
    }  
}
```

کلاس Object و متدهای آن

هر کلاس در جاوا به نوعی از نسل کلاس `java.lang.Object` به حساب می‌آید. اگر در تعریف یک کلاس، ارث بری اعلام نشود، به طور ضمنی ابرکلاس آن کلاس `Object` می‌باشد.

برای مثال `public class Circle extends Object{}` معادل است با `public class Circle{}`



کلاس `Object` پدر همه کلاس‌های جاوا است!

در شکل زیر می‌توانید متدهای موجود در کلاس Object را مشاهده کنید.

Method	Description
public final Class getClass()	returns the Class class object of this object. The Class class can further be used to get the metadata of this class.
public int hashCode()	returns the hashcode number for this object.
public boolean equals(Object obj)	compares the given object to this object.
protected Object clone() throws CloneNotSupportedException	creates and returns the exact copy (clone) of this object.
public String toString()	returns the string representation of this object.
public final void notify()	wakes up single thread, waiting on this object's monitor.
public final void notifyAll()	wakes up all the threads, waiting on this object's monitor.
public final void wait(long timeout)throws InterruptedException	causes the current thread to wait for the specified milliseconds, until another thread notifies (invokes notify() or notifyAll() method).
public final void wait(long timeout,int nanos)throws InterruptedException	causes the current thread to wait for the specified milliseconds and nanoseconds, until another thread notifies (invokes notify() or notifyAll() method).
public final void wait(long timeout,int nanos)throws InterruptedException	causes the current thread to wait for the specified milliseconds and nanoseconds, until another thread notifies (invokes notify() or notifyAll() method).
public final void wait()throws InterruptedException	causes the current thread to wait, until another thread notifies (invokes notify() or notifyAll() method).
protected void finalize()throws Throwable	is invoked by the garbage collector before object is being garbage collected.

toString()

این متد رشته‌ای را برمی‌گرداند که نمایش‌دهنده شی‌ای حاصل از کلاس مربوطه است. پیاده‌سازی پیش‌فرض این متد، رشته‌ای حاوی نام کلا مربوط به شی را برمی‌گرداند که به دنبال آن علامت @ و شماره شی آمده است.

```
Loan loan = new Loan();
```

```
System.out.println(loan.toString());
```

کد بالا عبارتی نظیر Laon@15037e5 را چاپ می‌کند.

این پیام اطلاعات زیادی در اختیار برنامه‌نویس قرار نمی‌دهد. درنتیجه در اکثر مواقع متد بازنویسی می‌شود تا رشته‌ای حاوی اطلاعات جامع‌تر در اختیار کاربر قرار دهد.

```
public class Circle extends GeometricObject {
    // Other methods are omitted

    /** Override the toString method defined in GeometricObject */
    public String toString() {
        return super.toString() + "\nradius is " + radius;
    }
}

//from class GeometricObject
/** Return a string representation of this object */
public String toString() {
    return "created on " + dateCreated+ "\ncolor: " + color + " and filled: " + filled;
}
```

بازنویسی این متد در زیرکلاس‌ها به چه صورت است؟

تبدیل رو به بالا یا Upcasting

تبدیل رو به بالا یا Upcasting : هرگاه متغیر ارجاعی کلاس والد به شیئی کلاس فرزند اشاره کند، می‌گوییم تبدیل رو به بالا انجام شده است.

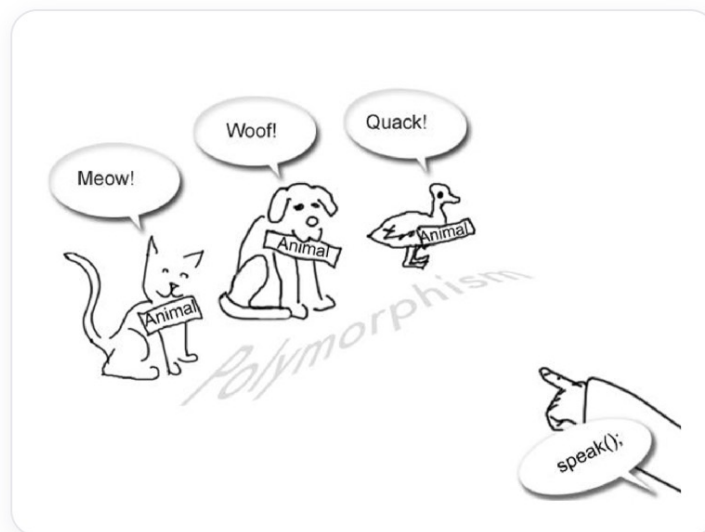
```
class A{}
```

```
class B{}
```

```
A a = new B();
```

مفهوم پلی مورفیسم

پلی مورفیسم زمان اجرا، فرآیندی است که فراخوانی یک متد بازنویسی شده در زمان اجرا و نه در زمان کامپایل انجام می‌شود. در این فرآیند، یک متد بازنویسی شده از طریق یک متغیر ارجاعی از کلاس والد (ریموت کنترل کلاس والد) فراخوانی می‌شود. تعیین اینکه کدام متد فراخوانی شود، به شی‌ای بستگی دارد که متغیر ارجاعی به عنوان آرگومان به آن اشاره می‌کند.



مثالی از پلی مورفیسم زمان اجرا

در این مثال دو کلاس Bike و Splender تعریف شده است، کلاس Splender از کلاس Bike ارث بری می‌کند و متد run() را بازنویسی می‌کند.

اینک، متد run() را از طریق متغیر ارجاعی کلاس والد فراخوانی می‌کنیم.

چون متغیر ارجاعی به شیئی زیرکلاس اشاره می‌کند و متد زیرکلاس، متد ابرکلاسش را بازنویسی کرده است، در زمان اجرا متد زیرکلاسش فراخوانی می‌شود.

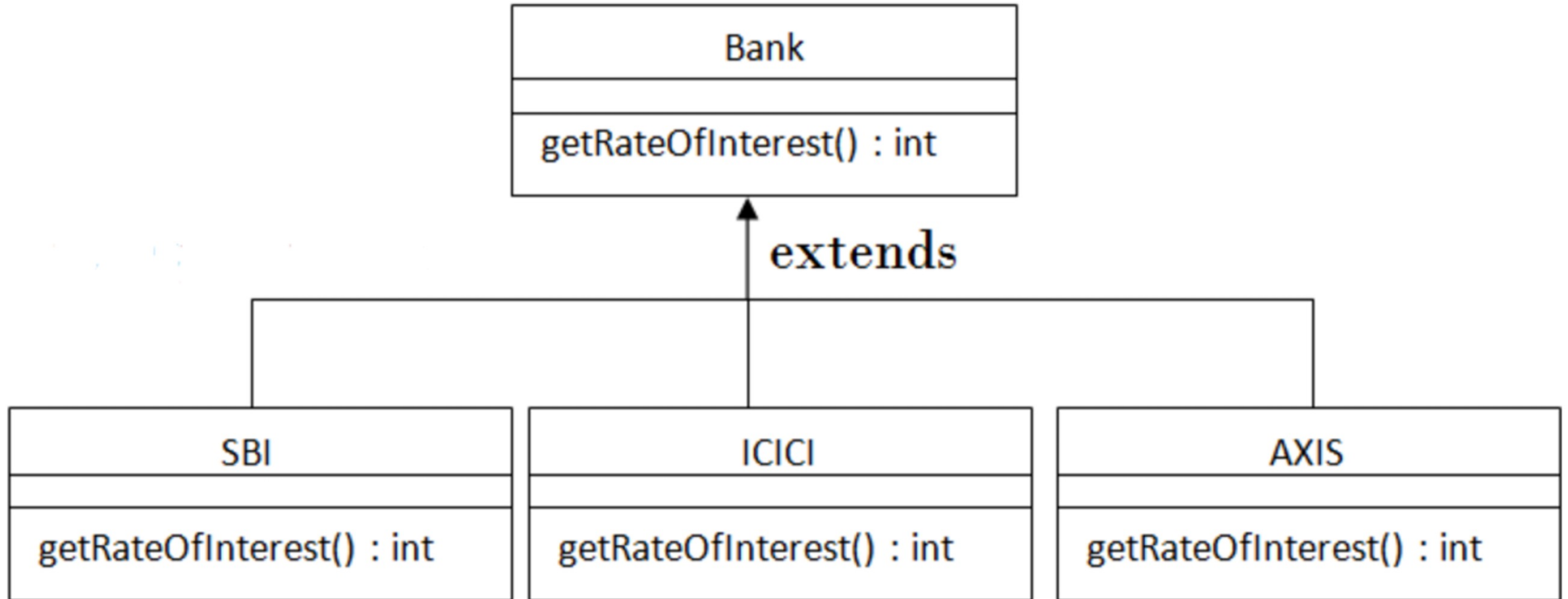
چون فراخوانی متد توسط JVM و نه کامپایلر انجام می‌شود، این فراخوانی به پلی مورفیسم زمان اجرا شهرت دارد.

```
class Bike{
    void run(){
        System.out.println("running");
    }
}
class Splender extends Bike{
    void run(){
        System.out.println("running safely with 60km");
    }

    public static void main(String[] args){
        Bike b = new Splender(); //upcasting
        b.run();
    }
}
```

مثالی دیگر

سناریو بانک را در نظر بگیرید و با استفاده از پلی مورفیسم متد `getRateOfInterest()` را فراخوانی کنید.



```
class Bank {  
    int getRateOfInterest() {  
        return 0;  
    }  
}
```

```
class SBI extends Bank {  
    int getRateOfInterest() {  
        return 8;  
    }  
}
```

```
class ICICI extends Bank {  
    int getRateOfInterest() {  
        return 7;  
    }  
}
```

```
class AXIS extends Bank {  
    int getRateOfInterest() {  
        return 9;  
    }  
}
```

```
class Test3 {  
    public static void main(String args[]) {  
        Bank b1 = new SBI();  
        Bank b2 = new ICICI();  
        Bank b3 = new AXIS();  
  
        System.out.println("SBI Rate of Interest: " + b1.getRateOfInterest());  
        System.out.println("ICICI Rate of Interest: " + b2.getRateOfInterest());  
        System.out.println("AXIS Rate of Interest: " + b3.getRateOfInterest());  
    }  
}
```


پلی مورفیسم زمان اجرا با فیلد داده‌ای

همیشه متدها هستند که بازنویسی می‌شوند و نه فیلدهای داده‌ای کلاس.

بنابراین پلی مورفیسم بر روی فیلدهای داده‌ای قابل دستیابی نخواهد بود.

در مثال زیر، هر دو کلاس دارای فیلد داده‌ای speedlimit هستند. دسترسی به این فیلد داده‌ای توسط متغیر ارجاعی کلاس والد است که به شی زیرکلاس اشاره می‌کند یا به عبارتی ریموت کنترل آن است.

چون ما به فیلد داده‌ای دسترسی داریم که بازنویسی نشده است و در اصل امکان بازنویسی آن وجود ندارد، همیشه فقط مقدار فیلد داده‌ای کلاس والد قابل دسترسی می‌باشد.

```
class Bike{
    int speedlimit = 90;
}

class Honda3 extends Bike{
    int speedlimit = 150;

    public static void main(String[] args){
        Bike obj = new Honda3();
        System.out.println(obj.speedlimit); //90
    }
}
```

مثالی از پلی مورفیسم زمان اجرا با ارث بری چند سطحی

```
class Animal{
    void eat(){
        System.out.println("eating");
    }
}

class Dog extends Animal{
    void eat(){
        System.out.println("eating fruits");
    }
}
```

```
class BabyDog extends Dog{
    void eat(){
        System.out.println("drinking milk");
    }

    public static void main(String[] args){
        Animal a1, a2, a3;
        a1 = new Animal();
        a2 = new Dog();
        a3 = new BabyDog();

        a1.eat();
        a2.eat();
        a3.eat();
    }
}
```

مثالی دیگر

از آنجایی که BabyDog متد eat() را بازنویسی نکرده است، متد eat() از کلاس Dog فراخوانی می‌شود.

```
class Animal{
    void eat(){
        System.out.println("animal is eating..");
    }
}

class Dog extends Animal{
    void eat(){
        System.out.println("dog is eating...")
    }
}

class BabyDog1 extends Dog(){
    public static void main(String[] args){
        Animal a = new BabyDog1();
        a.eat();
    }
}
```

مقید سازی ایستا و مقید سازی پویا

ارتباط دادن یک فراخوانی متد به بدنه متد، اصطلاحاً **مقید سازی (binding)** متد به پیاده سازی آن گفته می شود.

به طور کلی دو نوع مقید سازی داریم: **مقید سازی ایستا** یا به عبارتی مقید سازی زودهنگام و **مقید سازی پویا** که به آن مقید سازی دیرهنگام هم گفته می شود.



قبل از توضیح انواع مقیدسازی بهتر است به فهم نوع (type) فیلدها بپردازیم.

ابتدا نوع یک نمونه (instance type) را به طور دقیقتر بررسی می‌کنیم.

متغیرها دارای نوع هستند : هر متغیر نوعی دارد که می‌تواند **اصلی (primitive)** یا **ارجاعی (reference)** باشد.

```
int data = 30;
```

ارجاع‌ها نیز دارای نوع می‌باشند.

```
class Dog{  
public static void main(String[] args){ Dog d1; //Here d1 is a type of Dog}  
}
```

اشیا دارای نوع هستند: یک شی، نمونه‌ای از یک کلاس مشخص جاوا می‌باشد، اما در عین حال نمونه‌ای از کلاس والدش هم هست.

```
class Animal{  
  
class Dog extends Animal{  
public static void main(String[] args){ Dog d1 = new Dog() }  
}
```

مقیدسازی ایستا: هرگاه نوع یک شی در زمان کامپایل (توسط کامپایلر) مشخص شود، می‌گوییم **مقیدسازی ایستا** انجام شده است.

مقیدسازی پویا: هرگاه نوع شی در زمان اجرا مشخص شود، می‌گوییم **مقیدسازی پویا** انجام شده است.

مقیدسازی پویا به این شکل عمل می‌کند: فرض کنید شی o یک نمونه از کلاس‌های $C_1, C_2, \dots, C_{n-1}, C_n$ باشد، به طوری که C_1 یک زیرکلاس C_2 و C_2 یک زیرکلاس C_3 و ... و C_{n-1} یک کلاس C_n باشد. یعنی C_n عمومی‌ترین کلاس باشد و C_1 اختصاصی‌ترین کلاس باشد. در جاوا C_n همان کلاس `Object` است. اگر o یک متد p را فراخوانی کند، JVM به دنبال پیاده‌سازی متد p به ترتیب در $C_1, C_2, \dots, C_n$ می‌گردد تا زمانی که متد مربوطه را بیابد. به محض یافتن متد، جستجو خاتمه یافته و پیاده‌سازی شناسایی شده فراخوانی می‌شود.



```

/*مقیدسازی ایستا*/

class Dog{
    private void eat() {
        System.out.println("dog is eating...");
    }

    public static void main(String[] args){
        Dog d1 = new Dog();
        d1.eat();
    }
}

```

```

/*مقیدسازی پویا*/

class Animal{
    void eat() {
        System.out.println("animal is eating...");
    }
}

class Dog extends Animal{
    private void eat() {
        System.out.println("dog is eating...");
    }

    public static void main(String[] args){
        Animal a = new Dog();
        a.eat();
    }
}

```

در مثال بالا نوع شی نمی‌تواند در زمان کامپایل توسط کامپایلر مشخص شود، چون یک شی (نمونه) از کلاس Dog یک نمونه از کلاس Animal نیز می‌باشد. در نتیجه کامپایلر نوع دقیق آن را نمی‌شناسد.

تطبیق متد در مقایسه با مقیدسازی متد یا به عبارتی Method Matching vs. Binding

تطبیق امضای یک متد (signature) و مقیدسازی پیاده‌سازی یک متد، دو موضوع مختلف هستند.

کامپایلر تطبیق متد را براساس نوع پارامترها، تعداد و ترتیب پارامترها را در زمان کامپایل انجام می‌دهد.

یک متد ممکن است در چندین زیرکلاس بازنویسی شده باشد.

ماشین مجازی جاوا، به صورت پویا تشخیص می‌دهد که کدام پیاده‌سازی متد را در زمان اجرا در نظر بگیرد (مقیدسازی کند).

تبدیل اشیا (Casting Objects)

در جلسات اول گفتیم که عملگر casting برای تبدیل متغیرهای یک نوع اصلی به نوع اصلی دیگر استفاده می‌شود. این عملگر را می‌توان برای تبدیل یک شی از یک نوع کلاس به شی‌ای از کلاسی دیگر از طریق سلسله مراتب وراثتی مورد استفاده قرار داد. متد زیر را در نظر بگیرید.

```
public static void m(Object x){ System.out.println(x.toString()); }
```

در این متد یک متغیر ارجاعی از کلاس Object به عنوان پارامتر به متد ارسال می‌شود. دستور زیر شی new Student() را از نوع کلاس پدر جدش (Object) تعریف می‌کند.

```
Object o = new Student();
```

دستور بالا یک تبدیل رو به بالا Upward casting نامیده می‌شود و تبدیلی ضمنی (Implicit casting) است. این تبدیل در جاوا مجاز است چون نمونه از کلاس Student به طور خودکار یک نمونه از کلاس Object به حساب می‌آید.

سوالی مهم پیش می‌آید: چرا تبدیل لازم است؟

فرض کنید می‌خواهید ارجاع به شی `o` از کلاس `Object` را به متغیری از نوع `Student` با استفاده از دستور زیر انتساب دهید.

با این دستور، یک خطای کامپایلری رخ خواهد داد. سوال اینجاست: چرا دستور `Object o = new Student()` اجرا می‌شود اما `Student b = o` کامپایل نمی‌شود؟

دلیل این است که شی `Student` همیشه یک نمونه از کلاس `Object` به حساب می‌آید، اما یک شی از `Object` لزوماً یک نمونه از `Student` نیست. حتی اگر از نظر شما `o` واقعاً شی‌ای از `Student` باشد، کامپایلر آنقدر باهوش نیست که این مسئله را درک کند.

برای اینکه به کامپایلر بگوییم `o` یک شی از `Student` است، از تبدیل آشکار (`explicit`) استفاده می‌کنیم. قاعده مورد استفاده، مشابه قاعده تبدیل نوع اصلی می‌باشد (استفاده از دو پرانتز برای نوع شی‌ای که می‌خواهیم تبدیل را بر روی آن انجام دهیم).

```
Student b = (Student)o; //Explicit casting
```

تبدیل از کلاس والد به کلاس فرزند (Downcasting)

تبدیل آشکار یا همان Explicit در هنگام تبدیل ریموت کنترل یک شی از جنس کلاس والد به کلاس فرزندش استفاده می‌شود. این تبدیل همیشه موفقیت‌آمیز نمی‌باشد!

```
Apple x = (Apple)fruit;
```

```
Orange x = (Orange)fruit;
```

عملگر instanceof

از این عملگر به این هدف استفاده می‌کنیم تا تست کنیم آیا یک شی نمونه‌ای از کلاس موردنظر می‌باشد یا خیر؟

```
Object myObject = new Circle();  
  
...//Some lines of code  
  
/* Perform casting if myObject is an instance of Circle */  
if(myObject instanceof Circle){  
  
    System.out.println("The circle diameter is " + ((Circle)myObject).getDiameter());  
  
    ...  
}
```

مثال‌هایی از عملگر instanceof:

یک شی از کلاس فرزند، یک شی از کلاس والد نیز هست. برای مثال، اگر Dog از Animal ارث بری کند، شی‌ای از Dog هم توسط کلاس Dog و هم توسط Animal می‌تواند مورد ارجاع قرار بگیرد.

```
class Animal { }

class Dog1 extends Animal { //Dog inherits Animal
    public static void main(String[] args) {
        Dog1 d = new Dog1();
        System.out.println(d instanceof Animal); //true
    }
}
```

اگر عملگر instanceof را با متغیر ارجاعی استفاده کنیم که دارای مقدار null است، false بر خواهد گرداند.

مثال زیر را ببینید:

```
class Dog2 {  
    public static void main(String[] args) {  
        Dog2 d = null;  
        System.out.println(d instanceof Dog2); //false  
    }  
}
```

تبدیل رو به پایین (Downcasting) با عملگر instanceof

وقتی یک متغیر ارجاعی از کلاس فرزند شی‌ای از کلاس والد را کنترل کند، می‌گوییم تبدیل رو به پایین انجام شده است. اگر این تبدیل مستقیماً انجام شود، کامپایلر خطا خواهد گرفت.

```
Dog d = new Animal(); //Compilation error
```

اگر مستقیماً با typecasting انجام شود (گذاشتن کلاس فرزند در پرانتز)، دچار خطای مان اجرا خواهد شد!

```
Dog d = (Dog)new Animal();
```

```
//Compiles successfully but ClassCastException is thrown at runtime
```

اما به کمک عملگر instanceof و با بررسی نوع شی، تبدیل رو به پایین انجام خواهد شد.

```
class Animal{ }

class Dog3 extends Animal {
    static void method(Animal a) {
        if(a instanceof Dog3) {
            Dog3 d = (Dog3)a; //downcasting
            System.out.println("ok downcasting performed");
        }
    }

    public static void main(String[] args) {
        Animal a = new Dog3();
        Dog3.method(a);
    }
}
```


تبدیل رو به پایین (Downcasting) اگر از عملگر instanceof استفاده نکنیم

اگر نخواهیم از عملگر instanceof برای تبدیل رو به پایین استفاده کنیم، به شکل زیر عمل می‌کنیم:

```
class Animal { }
class Dog4 extends Animal {
    static void method(Animal a) {
        Dog4 = (Dog4)a; //downcasting
        System.out.println("ok downcasting performed");
    }

    public static void main(String[] args) {
        Animal a = new Dog4();
        Dog4.method(a);
    }
}
```

در این مثال چون a را از کلاس والد تعریف کردیم که به شی کلاس فرزند اشاره می‌کند، شی ایجاد شده، در واقع نمونه‌ای از کلاس Dog می‌باشد. در نتیجه در داخل method متغیر a بدون اشکال به متغیر Dog تبدیل می‌شود.

اما اگر دستور را به صورت زیر بنویسیم با خطای زمان اجرا رو به رو می‌شویم:

```
Animal a = new Animal();
```

```
Dog4.method(a);
```

```
//Now ClassException but not in case of instanceof operator
```

```

/* مثالی از کاربرد مهم عملگر instanceof */

interface printable{}

class A implements printable{
    public void a() {
        System.out.println("a method");
    }
}

class B implements printable{
    public void b() {
        System.out.println("b method");
    }
}

```

```

class Call{
    void invoke(printable p){//upcasting
        if(p instanceof A){
            A a = (A)p; //downcasting
            a.a();
        }
        if(p instanceof B){
            B b = (B)p; //downcasting
            b.b();
        }
    }
}

```

```

class Test4{
    public static void main(String[] args){
        printable p = new B();
        Call c = new Call();
        c.invoke(p);
    }
}

```

مثال پلی مورفیسم و casting

برنامه‌ای بنویسید که دو شی هندسی ایجاد کند: یک دایره، یک مستطیل

کافیست کلاسی جدید تعریف کنید و متد main را درون آن بنویسید. سپس از کلاس والد Object دو متغیر ارجاعی (object2, object1)، یکی به دایره و دیگری به مستطیل تعریف کنید.

سپس متد displayGeometricObject را برای دو شی جدید فراخوانی کنید.

پس از اتمام main متد displayGeometricObject را این گونه تعریف کنید: این متد استاتیک، پارامتری از نوع Object داشته باشد. اگر پارامتر ارسالی دایره بود، یک تبدیل رو به پایین انجام شده و متدهای مساحت و قطر دایره فراخوانی شوند. اگر پارامتر ارسالی مستطیل بود، متد مساحت آن فراخوانی شود.

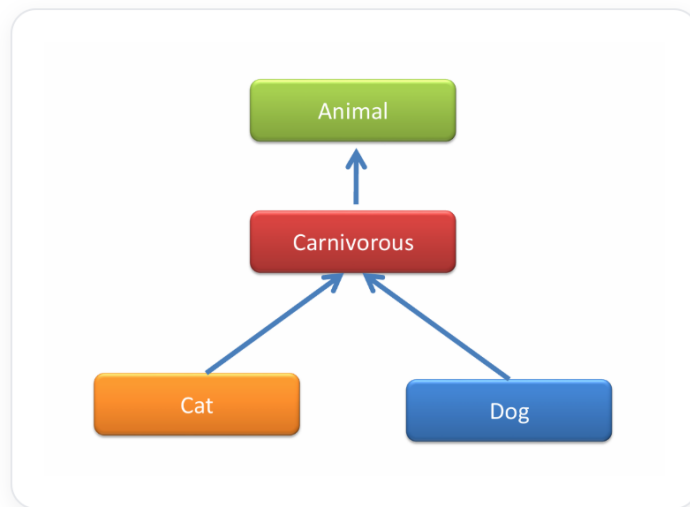
```
public class CastingDemo {

    /** Main method */
    public static void main(String[] args) {
        // Declare and initialize two objects
        Object object1 = new Circle4(1);
        Object object2 = new Rectangle1(1, 1);

        // Display circle and rectangle
        displayObject(object1);
        displayObject(object2);
    }

    /** A method for displaying an object */
    public static void displayObject(Object object) {
        if (object instanceof Circle4) {
            System.out.println("The circle area is " + ((Circle4)object).getArea());
            System.out.println("The circle diameter is " + ((Circle4)object).getDiameter());
        }
        else if (object instanceof Rectangle1) {
            System.out.println("The rectangle area is " + ((Rectangle1)object).getArea());
        }
    }
}
```

تبدیل رو به بالا / رو به پایین - مثال جانوران



به این نکته توجه کنید که: با تبدیل اشیا شما هویت واقعی شی را تغییر نمی‌دهید. بلکه به شی مربوطه **برچسب متفاوتی می‌زنید**.

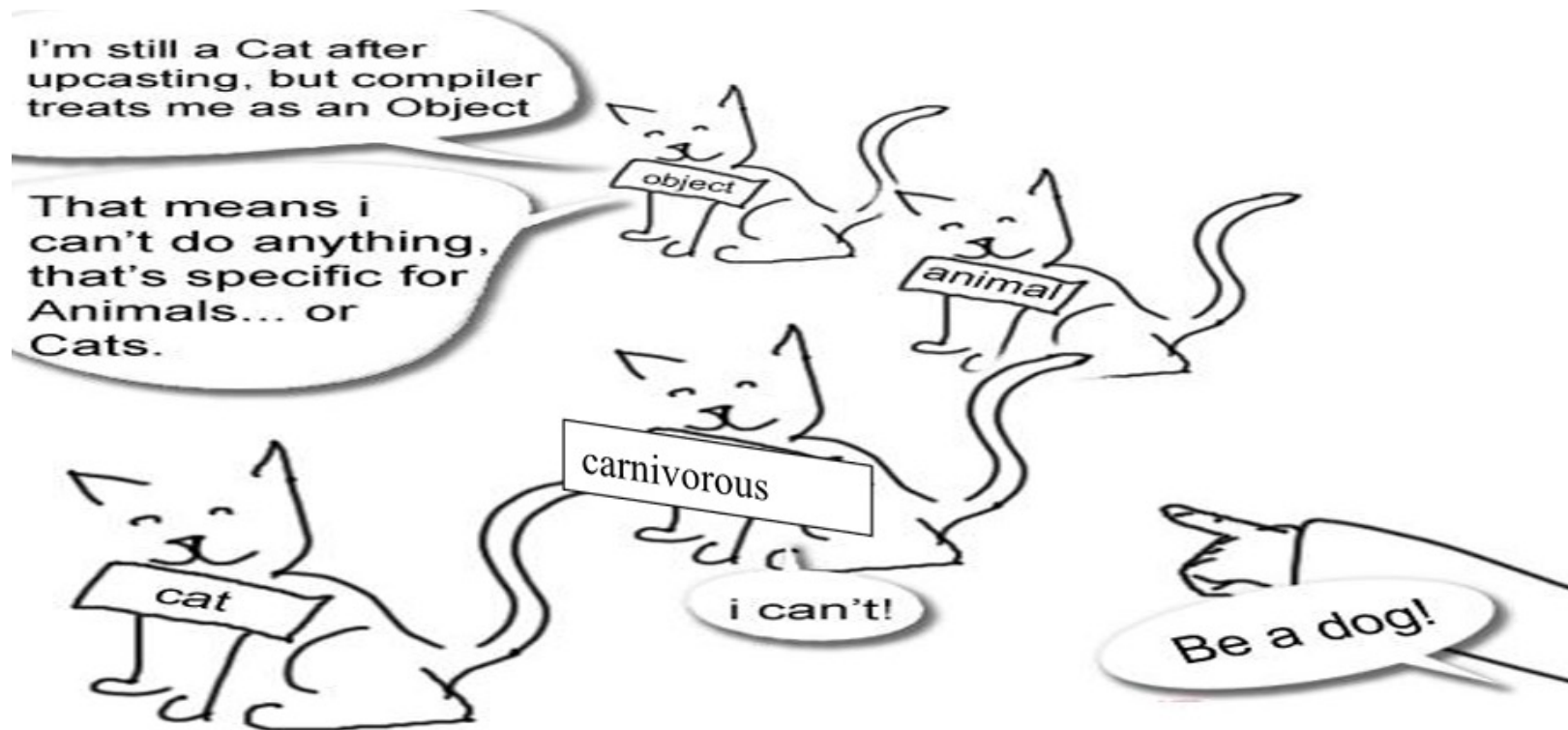
برای مثال، اگر شما یک Cat ایجاد کرده و بر روی آن تبدیل رو به بالا انجام دهید، این تبدیل سبب نمی‌شود که شی اولیه دیگر گربه (Cat) نباشد!

این شی همچنان گربه است. اما با او مانند هر جانور (Animal) دیگری رفتار خواهد شد و خصوصیات (فیلدهای) ویژه گربه پنهان می‌شود تا زمانی که دوباره به گربه تبدیل رو به پایین شود.

همانطور که در کد مربوط به این مثال مشاهده می‌کنید، Cat پس از تبدیل همچنان Cat باقی مانده و به جای آن یک شی گوشت‌خوار (Carnivorous) ایجاد نشده است. فقط برچسب گوشت‌خوار به آن زده شده که کاملاً مجاز است، چون یک گربه، یک گوشت‌خوار هم می‌باشد!

```
Cat c = new Cat();  
System.out.println(c);  
Carnivorous m = c; // upcasting  
System.out.println(m);  
  
/* This printed:  
Cat@a90653  
Cat@a90653 */
```

توجه داشته باشید اگرچه دو شی گوشت خوار هستند، گربه نمی تواند به سگ تبدیل شود. شکل زیر این مطلب را نشان می دهد:



خودکار انجام شدن تبدیل رو به بالا:

برای تبدیل رو به بالا نیاز نیست برنامه‌نویس به صورت دستی این کار را انجام دهد. برای مثال

`Carnivorous m = new Cat();` معادل است با `Carnivorous m = (Carnivorous)new Cat();`

اما تبدیل رو به پایین همیشه باید به صورت دستی انجام شود:

```
Cat c1 = new Cat();
```

```
Animal a = c1; //automatic upcasting to Animal
```

```
Cat c2 = (Cat) a; //manual downcasting back to a Cat
```

تفاوت تبدیل رو به بالا و تبدیل رو به پایین

چرا تبدیل رو به بالا به صورت خودکار انجام می‌شود اما تبدیل رو به پایین باید به طور دستی انجام شود؟

چون تبدیل رو به بالا هرگز با خطا رو به رو نخواهد شد. اما اگر مجموعه‌ای از جانوران مختلف داشته باشیم و بخواهیم با تبدیل رو به پایین همه آنها را به گربه تبدیل کنیم، در زمان اجرا این احتمال وجود دارد که برخی از این جانوران واقعا گربه نباشند (مثلا اردک باشند) و در نتیجه فرآیند تبدیل رو به پایین با پیام `ClassCastException` با شکست مواجه خواهد شد.

به همین دلیل است که از عملگر `instanceof` استفاده می‌کنیم تا تست کند آیا شی مربوط، نمونه‌ای از کلاس گربه می‌باشد یا خیر.

مثال زیر را در نظر بگیرید :

```
Cat c1 = new Cat();

Animal a = c1; //upcastingto Animal

if(a instanceof Cat){ // testing if the Animal is a Cat

System.out.println("It's a Cat!

Now i can safely downcast it to a Cat, without a fear of failure");

Cat c2 = (Cat)a;}
```

مشخص است که تبدیل همواره در هر دو جهت قابل انجام نمی‌باشد. اگر با دستور "new Carnivorous()" یک شی گوشت‌خوار ایجاد کرده باشید شی مورد نظر نمی‌تواند مستقیماً به یک سگ یا یک گربه تبدیل شود، چون در واقع هیچ یک از آنها نمی‌باشد!

برای مثال تبدیل زیر نادرست است.

```
Carnivorous m = new Carnivorous ();
```

```
Cat c = (Cat)m ;
```

این کد با پیام زمان اجرای java.lang.ClassCastException رو به رو می‌شود چون به زور می‌خواهیم گوشت‌خواری را که لزوماً گربه نیست، به گربه تبدیل کنیم.

در حالت کلی برای اینکه بدانیم چه موقع از تبدیل مناسب استفاده کنیم، این سوال را از خود بپرسیم : آیا گربه یک گوشت‌خوار است؟ بله، هست. پس تبدیل گربه به گوشت‌خوار قابل انجام است!

دوباره بپرسیم: آیا یک گوشت‌خوار یک گربه است؟ پاسخ، خیر است. پس تبدیل گوشت‌خوار به گربه نمی‌تواند انجام بشود.

خودمون رو بسنجیم

این بخش برای این طراحی شده که در پایان مطالعه این اسلاید، بتونی خودت رو محک بزنی و ببینی آیا مفاهیم رو به خوبی یاد گرفتی یا نه. سوالات زیر رو مرور کن و سعی کن بدون نگاه کردن به متن درس، به اون ها پاسخ بدی.

- مهم‌ترین کاربرد عملگر instanceof توی چیه؟
- موقع بازنویسی متدهای ابرکلاس در زیرکلاس های اون باید حواسمون به چه نکاتی باشه؟
- تفاوت بین مقیدسازی ایستا و پویا در چه چیزیه؟

پایان

در صورت هرگونه سوال یا پیشنهاد میتونید با من
در ارتباط باشید (:

gmail: sarashiri0906@gmail.com